Golden Bike Shop Integrated System

Final Design Package: defending our significant decisions

Team Kickstarters · Angela Colton · Aela Mason · Joseph Frongillo

MSSE601 · Week 8 · October 2026

One shared view across three domains

The problem

- Retail, rentals and service run on separate systems with no real-time view across stores
- Managers can't see stock at other stores, whether a rental bike is truly available, or how repairs affect fleet and bay capacity
- Cost of doing nothing: bikes shown available that aren't, missed sales, idle service bays

We integrate

- Shared inventory, 4 stores
- Rental fleet status + telemetry
- Workshop bookings & capacity
- Role-based views

We don't replace

- Point-of-sale & payments
- Telemetry hardware
- Workshop management system
- E-commerce storefront

4
stores sharing inventory

~400
rental bikes, incl. e-bikes

3
business areas: retail, rental, service

$65K
budget over 8 weeks

Process: evolutionary lifecycle with Agile practices

Waterfall

Rejected

Assumes stable requirements up front. Too much was unknown: what counts as "available," how integrations behave, what each manager needs.

Rapid prototyping

Used inside Week 4

Great for early feedback on screens, but can't carry requirements, architecture, testing and deployment on its own.

Evolutionary + Agile

Selected

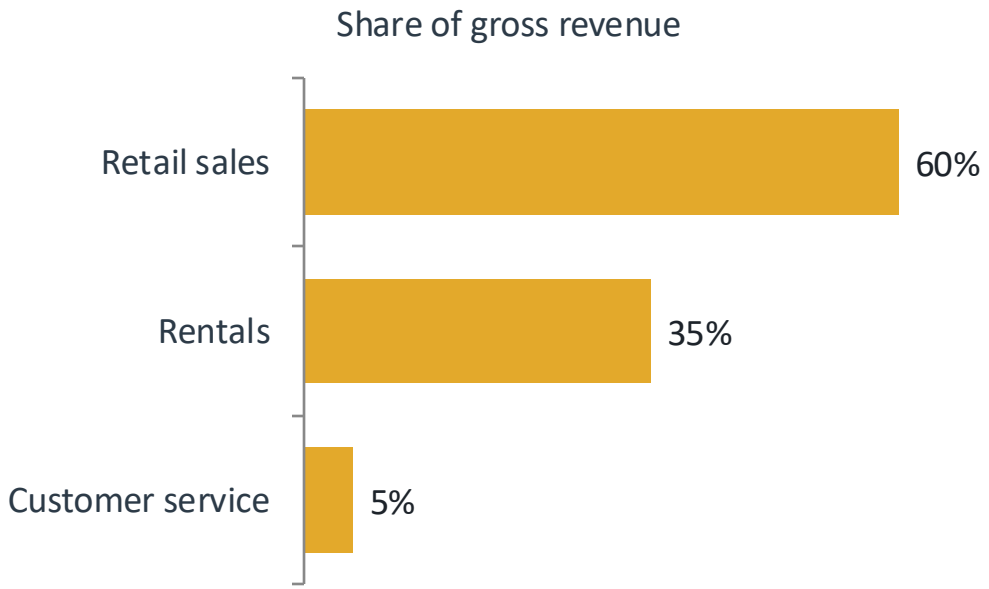
Deliver in increments, review with stakeholders each iteration, and fix wrong assumptions early.

What we gave up: upfront predictability of scope and schedule.

Team disagreement: Waterfall was proposed first, when we thought this was a new build. Once we saw it was an integration project with open questions, the team agreed on Agile.

We'd revisit if: requirements became fixed, stakeholders became unavailable for feedback, or the sponsor demanded full upfront approval.

Requirements: revenue set the priority order



13 user stories, each with testable acceptance criteria, ranked by business value, risk and dependency

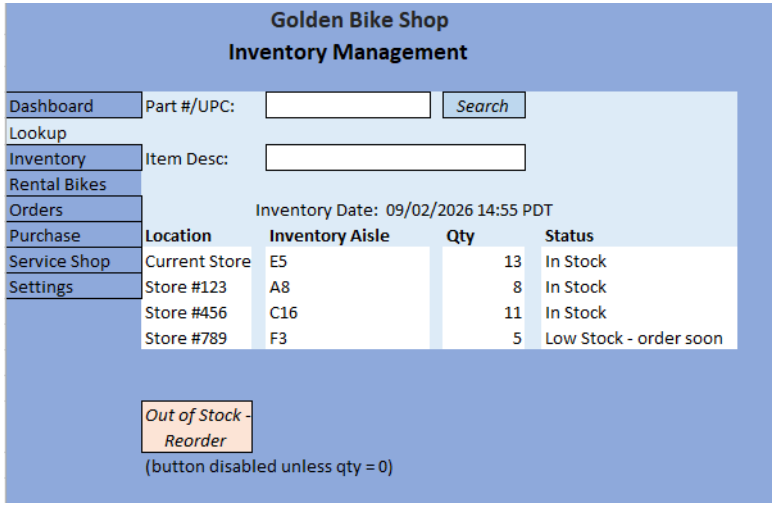
Highest priority first

1. **REQ-001/002** Look up any product's stock at all four stores; manager inventory view
2. **REQ-003/004/005** Set rental bike status; rent only when "available"; filter by type
3. **REQ-006/007** Book service bays with automatic conflict checks
4. **REQ-008/009** Online in-stock check; historical sales for managers
5. **REQ-010 to 013** Customer bike status, e-bike telemetry, rental history

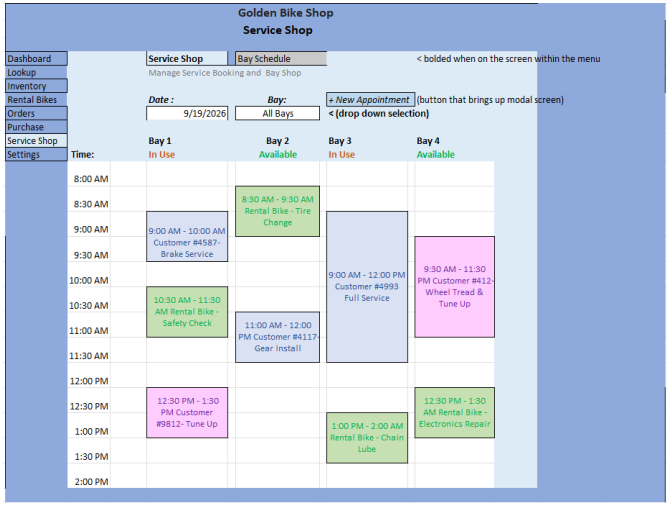
Non-functional requirements (NFR-001 to 007)

Inventory synced in 2 s	**Any approved browser**	**Role-based access**	**Documented & maintainable**	**80% finish tasks unaided**	**99.5% uptime, 9am-7pm**

UI: simple screens built around three journeys



Inventory lookup across four stores



Service bay calendar with conflict check

Retail manager

Checks stock and reorders

Rental employee

Rents only available bikes

Lead service staff

Books bays without overlaps

Accessibility: every color is paired with text, all controls work by keyboard, and error messages say how to fix the problem.

We compared against AI layouts

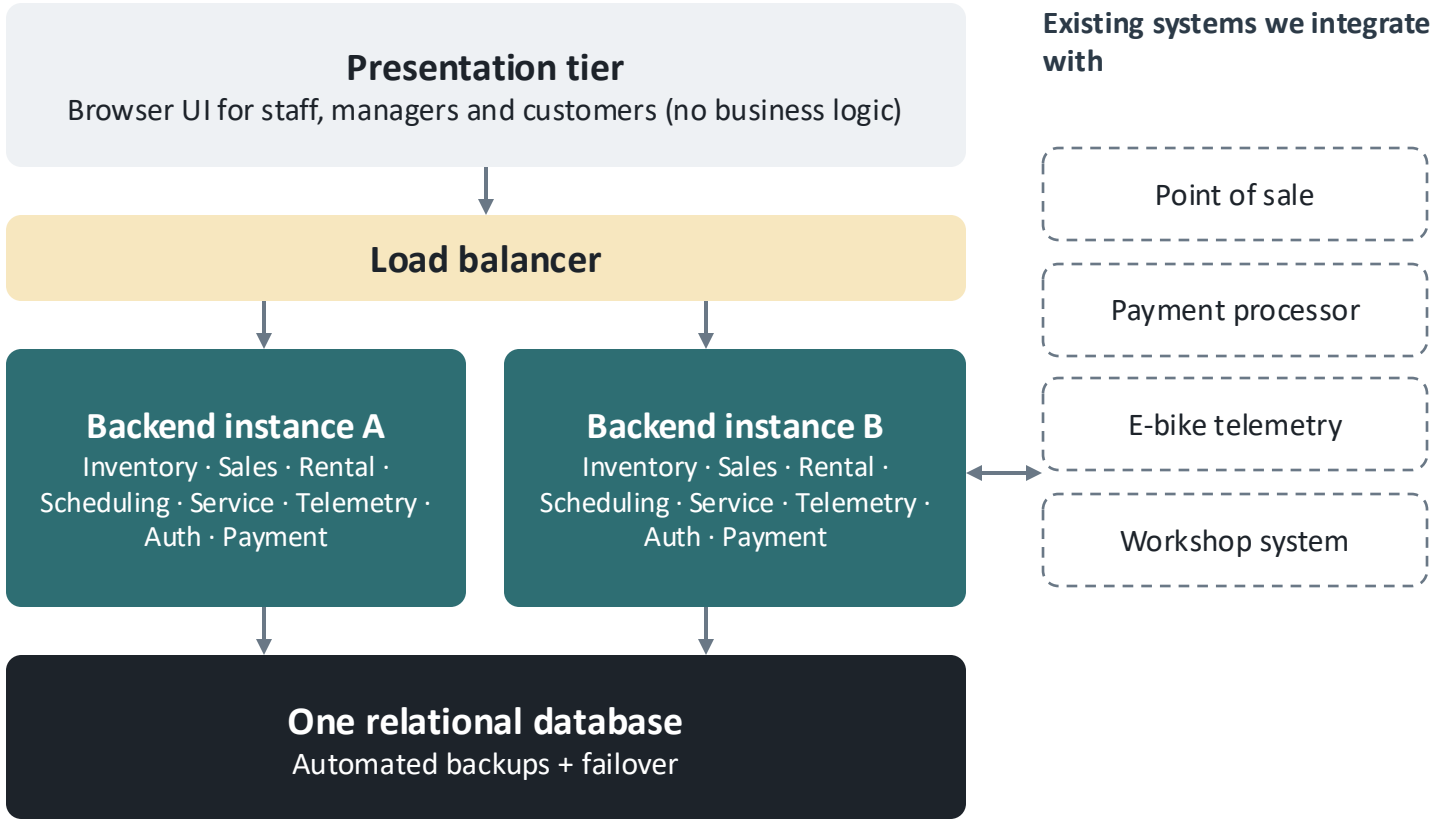
Adopted

- Inventory date stamp
- Filters & navigation menu
- Out-of-stock indicators
- Serial numbers in View Details

Rejected

- Map dashboards (not needed yet)
- Large icons for every bike
- Split-screen, crowded screens

Architecture: N-tier monolith, deployed with redundancy



Decisions and trade-offs

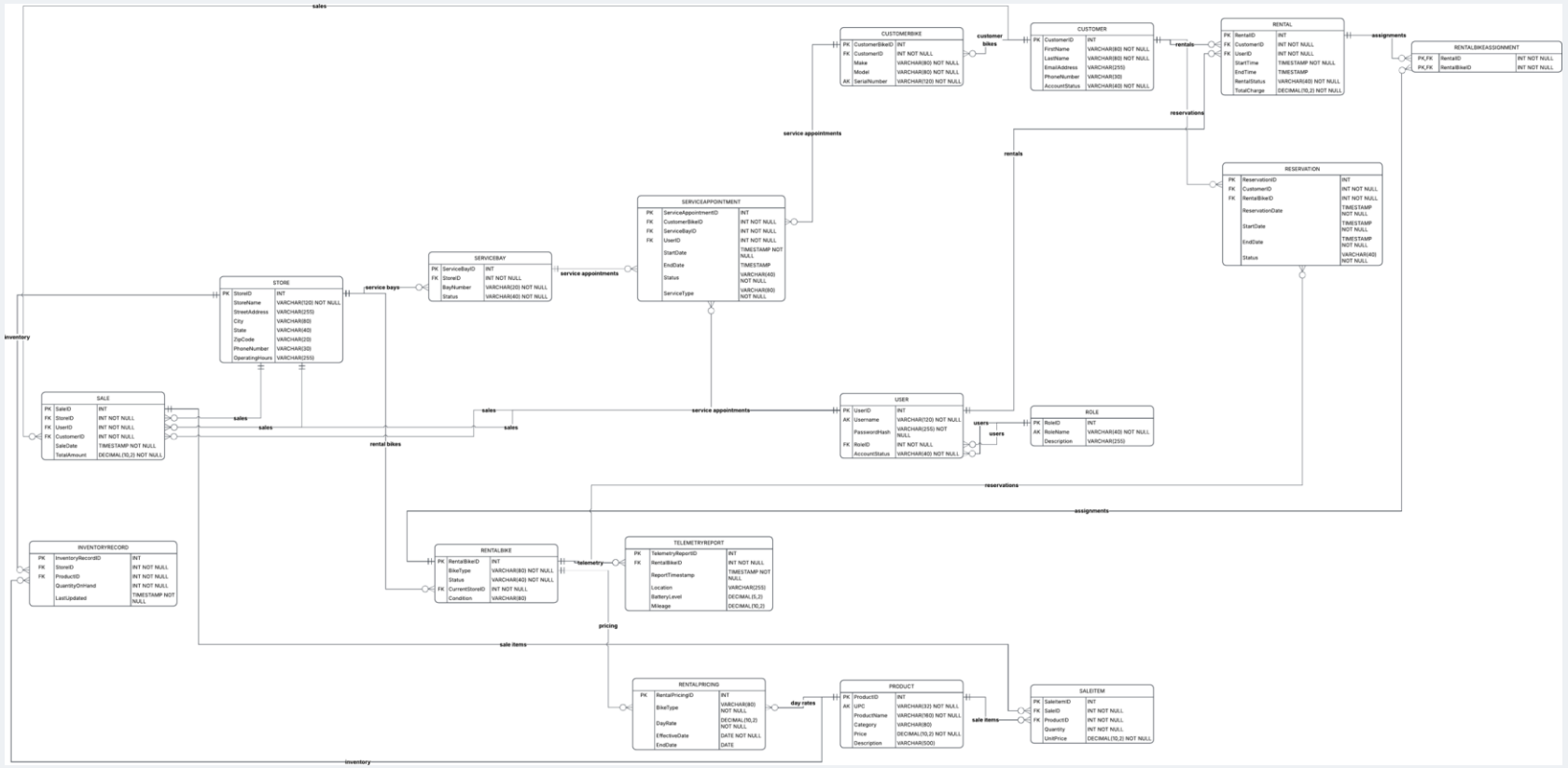
N-tier over backend-for-frontend.
Requirements describe one unified system; growth is expected to be small.

One database, one deployable backend.
The data is heavily linked; separate apps would add sync complexity.

Synchronous writes.
Changed NFR-001 from 1 minute to 2 seconds: no stale counts.

Gave up: scaling or surviving each domain independently.

ERD: 17 tables connecting retail, rental and service



Retail

Store · Product · InventoryRecord · Sale · SaleItem

Rental

RentalBike · Rental · RentalBikeAssignment · RentalPricing · Reservation · TelemetryReport

Service

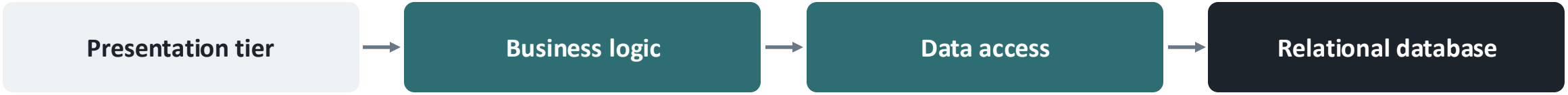
CustomerBike · ServiceBay · ServiceAppointment

People & access

Customer · User · Role

Primary keys, foreign keys and one-to-many relationships for every persisted object in the Week 5 object model

Data design: relational



How data moves, traced for three scenarios: create a reservation, start a rental, close a rental

1 **SQL over NoSQL**

Keys and referential integrity tie bikes, rentals, customers and bays together. We'd revisit if telemetry volume explodes.

2 **Added RentalPricing**

AI's model had no pricing, which showed us ours didn't either. Active rentals keep their original rate.

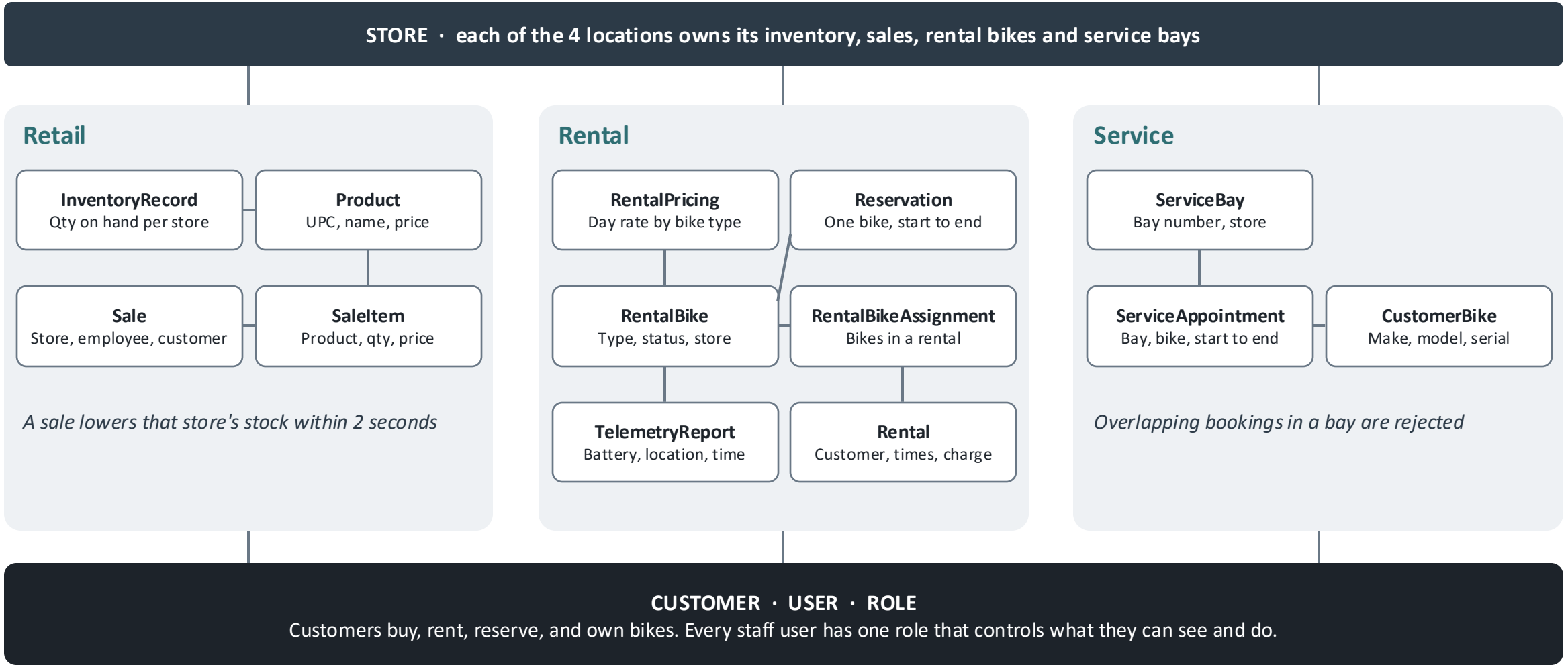
3 **One bike per reservation**

We rejected AI's multi-bike bridge table; it would have broken the approved UI and architecture.

4 **No "offline" bike status**

AI marked bikes offline when telemetry went stale, which hides their real status. We show a warning instead.

Data model: how the business fits together



Each line is a one-to-many relationship. 17 tables in total.

Quality: test hardest where failure costs most

1. **Product search, all four stores** — REQ-001
2. **Rent only available bikes** — REQ-004
3. **Bay double-booking rejected** — REQ-006
4. **Inventory updates within 2 s** — NFR-001
5. **Employee blocked from manager API (403)** — NFR-003

Gaps AI found that we accepted

- Two clerks renting the same bike at once
- Telemetry outage or bad data
- Unauthorized access across every role
- Changes in one domain reaching the others

Gaps we rejected, and the risk we accept

- Enterprise-scale load testing: 4 stores and ~400 bikes don't justify it
- Lighter testing of account setup and profiles: rare, simple, internal-only

Definition of Done: acceptance criteria met, automated tests pass, peer-reviewed, docs updated, and demonstrated to the stakeholder. Gregg Bishop gives final approval.

Deployment: three environments, gated releases

1 Development

Developers build and run unit tests

Gate: peer code review + unit tests pass

2 QA / Staging

Its own non-production database; integration, regression and UAT

Gate: stakeholder UAT + owner approval

3 Production

Two backend instances, one database with backups

Rolling update, one instance at a time

How we release

- Deploy outside store hours (9am-7pm); users see a maintenance notice at log-in (NFR-007)
- Update one backend instance at a time, so the shop stays online during releases
- Database changes stay backward-compatible, so we can roll back to the last build

Go-live plan

- Pilot at one store, then roll out to all four before the seasonal spike
- Train staff by role (Week 8), with Marcus as the first trained user
- Existing POS and rental systems stay live as the fallback during cutover

Operations: keep it running, keep it fixable

Monitor

- Alert on any response over 3 seconds
- Track uptime against 99.5% (about 3 minutes a day)
- Flag stale telemetry and unhealthy e-bikes on a scheduled pull

Back up and recover

- Automated database backups with failover
- Test a full restore before go-live, then regularly
- Load balancer routes around a failed instance

Support escaped defects

- Severity: Critical, High, Medium, Low
- Critical (rentals, inventory, access): hotfix outside the sprint
- Re-test, run regression, then the product owner closes it

Maintain (NFR-004)

- Setup, deployment and component docs kept current
- Full regression suite on every change (monolith)
- Every fixed bug gets an automated test